

Simulação e Teste de Software (CC8550)

Aula 6 – Técnicas Caixa-Preta: Partição de Equivalência e Valor-Limite

Prof. Luciano Rossi

Ciência da Computação
Centro Universitário FEI

2º Semestre de 2026

- ❶ Fundamentos e Taxonomia de Testes de Software
- ❷ Ciclo de Vida de Teste de Software (STLC)
- ❸ Planejamento Mestre de Testes e Análise de Risco
- ❹ Introdução à Automação de Testes com pytest
- ❺ Teste de Unidade Avançado (FIRST, padrão AAA)
- ❻ **Técnicas Caixa-Preta: Partição de Equivalência e Valor-Limite**
- ❼ Técnicas Caixa-Preta: Teste Combinatorial e Property-Based
- ❽ Técnicas Caixa-Branca: Critérios de Cobertura Estrutural
- ❾ Testes de Mutação e Avaliação da Eficácia de Suítes de Teste
- ❿ Test-Driven Development (TDD)
- ⓫ Teste de Integração, Test Doubles e Níveis de Teste
- ⓬ Teste Orientado a Objetos e de Componentes

Na aula 5 vimos os princípios FIRST e o padrão AAA para escrever testes de unidade confiáveis. Hoje aprendemos a escolher casos de teste de forma sistemática, sem testar tudo:

- ❶ O problema da exaustividade
- ❷ Teste caixa-preta
- ❸ Classes de equivalência
- ❹ Análise de valor-limite
- ❺ Combinando as duas técnicas
- ❻ Tabela de decisão
- ❼ Exercícios de fixação

Problema da Exaustividade

Quantos Casos de Teste Existem?

Contexto

Uma função recebe 3 parâmetros inteiros de 32 bits. Quantos casos de teste diferentes existem, considerando todas as combinações possíveis de entrada?

$$2^{32} \times 2^{32} \times 2^{32} = 2^{96} \approx 7,9 \times 10^{28}$$

Um número inimaginável

Mais casos de teste do que estrelas estimadas no universo observável. Testar cada combinação individualmente não é uma opção.

Testar Tudo é Inviável

Conceito

Se cada teste levasse 1 microssegundo para rodar, o tempo total para cobrir os 2^{96} casos seria de aproximadamente 2,5 quatrilhões de anos, muito além da idade do universo.

Conclusão

Teste exaustivo é impossível na prática. Precisamos de técnicas para selecionar um conjunto pequeno de casos que ainda assim represente bem o comportamento do sistema.

Teste Caixa-Preta

Testando pela Especificação

Definição

Teste caixa-preta

Teste baseado na especificação do sistema, sem acesso ao código-fonte. Foca no comportamento externo: o que entra, o que deve sair.

Complementa o teste caixa-branca, que examina a estrutura interna do código.

Onde o Caixa-Preta se Aplica

Conceito

Requisitos funcionais

Verifica se o sistema faz o que a especificação promete, sem depender de como foi implementado por dentro.

APIs

Testa o contrato de entrada e saída exposto publicamente: é justamente essa fronteira externa que a caixa-preta enxerga.

Teste de aceitação

Valida da perspectiva de quem vai usar o sistema, que também não vê nem precisa ver o código.

Analogia

Testar um carro sem abrir o capô: verificar se aceleração, freios e direção funcionam, sem saber como o motor foi construído por dentro.

Classes de Equivalência

Particionando o Domínio de Entrada

Conceito

A ideia: dividir o domínio de entrada em partições, chamadas classes de equivalência, onde todos os elementos de uma mesma classe levam o sistema a se comportar de forma equivalente.

Consequência prática

Se um elemento de uma classe revela um defeito, os demais elementos da mesma classe tendem a revelar o mesmo defeito. Basta testar um representante de cada classe.

- **Classes válidas:** entrada aceita, comportamento normal esperado
- **Classes inválidas:** entrada rejeitada, erro tratado

Identificando Classes: Intervalos Numéricos

Diagrama

Uma classe válida para o intervalo previsto na especificação, mais uma inválida abaixo do limite e outra acima.



Identificando Classes: Conjuntos, Booleanos e Strings

Definição

Conjuntos e enumerações

Uma classe válida para cada valor aceito, mais uma inválida agrupando os valores não previstos.

Condições booleanas

Uma classe para o caso verdadeiro, outra para o caso falso.

Strings

Por tamanho: vazia, normal ou acima do limite. Por conteúdo: caracteres permitidos ou não.

Exemplo: Desconto por Idade

Exemplo

Especificação

- 0 a 17 anos: sem desconto
- 18 a 64 anos: 10% de desconto
- 65 anos ou mais: 20% de desconto
- Idade negativa ou não numérica: erro

Quatro regras na especificação sugerem cinco classes de equivalência: uma para cada faixa válida, mais duas para as entradas inválidas.

Exemplo: Desconto por Idade

Classes de Equivalência

Classe	Intervalo	Tipo
CE1	idade < 0	Inválida
CE2	$0 \leq \text{idade} \leq 17$	Válida (0%)
CE3	$18 \leq \text{idade} \leq 64$	Válida (10%)
CE4	idade ≥ 65	Válida (20%)
CE5	não numérico	Inválida

Um representante por classe já cobre a especificação inteira: por exemplo, -5, 10, 30, 70 e "abc", cinco casos no lugar de milhares.

Exemplo: Desconto por Idade

⌕ Código

```
@pytest.mark.parametrize("idade,desconto", [
    (-5, None),      # CE1: idade negativa
    (10, 0.00),      # CE2: crianca
    (30, 0.10),      # CE3: adulto
    (70, 0.20),      # CE4: idoso
], ids=["negativa", "crianca", "adulto", "idoso"])
def test_calcular_desconto_idade(idade, desconto):
    if desconto is None:
        with pytest.raises(ValueError):
            calcular_desconto(idade)
    else:
        assert calcular_desconto(idade) == desconto
```

Exemplo: Desconto por Idade

⚡ Código: Entrada Não Numérica

```
def test_calcular_desconto_idade_tipo_invalido():  
    with pytest.raises(TypeError): # CE5: nao numerico  
        calcular_desconto("abc")
```

CE1 (idade negativa) e CE5 (tipo não numérico) são falhas diferentes: uma é valor fora da faixa, a outra é tipo errado. Por isso viram exceções diferentes, `ValueError` e `TypeError`, e testes separados.

Análise de Valor-Limite

Por Que os Bugs Vivem nas Fronteiras

Conceito

Observação empírica: defeitos se concentram nas fronteiras das classes de equivalência, não no meio delas.

- Erros de comparação: $<$ usado no lugar de $\leq$
- Erros off-by-one: contagem que começa ou termina um a mais ou a menos
- Limites mal definidos na especificação ou na implementação

Princípio

Além de testar um representante de cada classe, teste também os valores exatamente na fronteira e logo ao lado dela.

Regra dos Valores-Limite

Diagrama

Para um intervalo válido $[A, B]$, testar os seis valores ao redor das duas fronteiras:



Seis valores: $A-1$, A , $A+1$, $B-1$, B , $B+1$

Exemplo: Sistema de Notas

Exemplo

Especificação: uma nota é válida quando está entre 0,0 e 10,0, incluindo os dois extremos.

Classes de equivalência

- CE1: nota $< 0,0$ (inválida)
- CE2: $0,0 \leq \text{nota} \leq 10,0$ (válida)
- CE3: nota $> 10,0$ (inválida)

CE2 tem dois extremos, 0,0 e 10,0: cada um vira uma fronteira própria a testar.

Exemplo: Sistema de Notas

Diagrama

A mesma regra dos seis valores, agora com os extremos concretos do exemplo no lugar de A e B:



Exemplo: Sistema de Notas

Valores-Limite

Valor	Classe	Status
-0,1	CE1	Inválido
0,0	CE2	Válido (mínimo)
0,1	CE2	Válido
9,9	CE2	Válido
10,0	CE2	Válido (máximo)
10,1	CE3	Inválido

Seis valores cobrem as duas fronteiras de CE2 (mínimo e máximo) mais um representante de cada classe inválida vizinha.

Exemplo: Sistema de Notas

🔗 Código

```
@pytest.mark.parametrize("nota,valida", [
    (-0.1, False), (0.0, True), (0.1, True),
    (9.9, True), (10.0, True), (10.1, False),
], ids=["abaixo_min", "min", "acima_min",
        "abaixo_max", "max", "acima_max"])
def test_validar_nota(nota, valida):
    if valida:
        assert validar_nota(nota) is True
    else:
        with pytest.raises(ValueError):
            validar_nota(nota)
```

Combinando Classe e Limite

Estratégia Completa de Seleção

Conceito

Passo a passo

- 1) Identifique as classes de equivalência, válidas e inválidas
- 2) Para cada classe válida, teste um valor do meio e os valores-limite das suas fronteiras
- 3) Para cada classe inválida, teste um valor que viole a restrição e o valor-limite adjacente à fronteira

Resultado

Cobertura abrangente da especificação com um número pequeno e sistemático de casos de teste.

Exemplo: Temperatura de Operação

Exemplo

Especificação

A faixa segura de operação de um equipamento é -10°C a 50°C , incluindo os dois extremos. Fora dela, o sistema deve rejeitar a leitura.

Classes de equivalência

- CE1: $T < -10^{\circ}\text{C}$ (inválida)
- CE2: $-10^{\circ}\text{C} \leq T \leq 50^{\circ}\text{C}$ (válida)
- CE3: $T > 50^{\circ}\text{C}$ (inválida)

Exemplo: Temperatura de Operação

Representante e Valores-Limite

Valor	Classe	Status
-11°C	CE1	Inválido
-10°C	CE2	Válido (mínimo)
-9°C	CE2	Válido
20°C	CE2	Válido (representante do meio)
49°C	CE2	Válido
50°C	CE2	Válido (máximo)
51°C	CE3	Inválido

Sete casos aplicam os três passos juntos: um representante do meio de CE2, mais os seis valores-limite das duas fronteiras.

Exemplo: Temperatura de Operação

⌕ Código

```
@pytest.mark.parametrize("temp,valida", [
    (-11, False), (-10, True), (-9, True),
    (20, True), (49, True), (50, True), (51, False),
], ids=["abaixo_min", "min", "acima_min", "meio",
        "abaixo_max", "max", "acima_max"])
def test_temperatura_operacao(temp, valida):
    if valida:
        assert validar_temperatura(temp) is True
    else:
        with pytest.raises(ValueError):
            validar_temperatura(temp)
```

Tabela de Decisão

Múltiplas Condições, Muitas Regras

Conceito

Classes de equivalência e valor-limite tratam uma condição de cada vez. Quando o comportamento depende da **combinação** de várias condições independentes, outra técnica caixa-preta se aplica melhor:

- Múltiplas condições independentes, cada uma binária
- Lógica de negócio com várias regras de decisão
- Comportamento que muda conforme a combinação das condições, não de cada uma isolada

Estrutura da Tabela de Decisão

Definição

Estrutura

- **Condições:** as entradas ou critérios avaliados, cada um verdadeiro ou falso
- **Ações:** os resultados ou comportamentos esperados
- **Regras:** cada combinação de condições, associada à ação correspondente

Benefício

Enumerar a tabela inteira garante que nenhuma combinação relevante de condições fica de fora dos testes.

Exemplo: Aprovação de Empréstimo

Exemplo

Especificação

Aprovar o empréstimo somente se idade ≥ 18 e renda $\geq$ R\$3000 e score ≥ 600 . Rejeitar em qualquer outro caso.

Três condições binárias geram $2^3 = 8$ combinações possíveis: a tabela de decisão precisa cobrir as oito.

Exemplo: Aprovação de Empréstimo

Tabela de Decisão

Condição	R1	R2	R3	R4	R5	R6	R7	R8
Idade ≥ 18 ?	S	S	S	S	N	N	N	N
Renda ≥ 3000 ?	S	S	N	N	S	S	N	N
Score ≥ 600 ?	S	N	S	N	S	N	S	N
Aprovar	X							
Rejeitar		X	X	X	X	X	X	X

8 regras = 2^3 combinações, todas relevantes

Exemplo: Aprovação de Empréstimo

⌕ Código

```
@pytest.mark.parametrize("idade,renda,score,aprovado", [
    (25, 4000, 700, True),    (25, 4000, 500, False),
    (25, 2500, 700, False),  (25, 2500, 500, False),
    (17, 4000, 700, False),  (17, 4000, 500, False),
    (17, 2500, 700, False),  (17, 2500, 500, False),
]) # R1 a R8, na ordem da tabela
def test_aprovar_emprestimo(idade, renda, score, aprovado):
    resultado = aprovar_emprestimo(idade, renda, score)
    assert resultado == aprovado
```

Reduzindo a Tabela

💡 Conceito

Em algumas regras, uma condição deixa de importar porque outra condição já decide o resultado sozinha. Relembrando a tabela do empréstimo:

Condição	R1	R2	R3	R4	R5	R6	R7	R8
Idade $\geq$ 18?	S	S	S	S	N	N	N	N
Renda $\geq$ 3000?	S	S	N	N	S	S	N	N
Score $\geq$ 600?	S	N	S	N	S	N	S	N
Aprovar	X							
Rejeitar		X	X	X	X	X	X	X

Don't care (X): o valor dessa condição, verdadeiro ou falso, não muda a ação da regra.

Empréstimo Simplificado

Tabela Reduzida

Condição	R1	R2	R3	R4
Idade $\geq$ 18?	S	S	S	N
Renda $\geq$ 3000?	S	S	N	X
Score $\geq$ 600?	S	N	X	X
Aprovar	X			
Rejeitar		X	X	X

R4: se idade < 18 , renda e score não importam. **R3:** se renda < 3000 , score não importa. Redução: 8 regras viram 4.

Exemplo: Backup Automático

Exemplo

Especificação

Um aplicativo envia backup automático quando o dispositivo está conectado ao Wi-Fi **ou** carregando, **e** a bateria está acima de 20%.

Diferente do empréstimo: ali as três condições eram ligadas só por E. Aqui duas condições se combinam por OU antes de entrar num E com a terceira.

Exemplo: Backup Automático

Tabela de Decisão

Condição	R1	R2	R3	R4	R5	R6	R7	R8
Wi-Fi?	S	S	S	S	N	N	N	N
Carregando?	S	S	N	N	S	S	N	N
Bateria > 20%?	S	N	S	N	S	N	S	N
Backup	X		X		X			
Não		X		X		X	X	X

8 regras: OU decide entre Wi-Fi e carregando, E exige bateria acima de 20%

Reduzindo com OU e E

Conceito

Condição	Ra	R1	R3	R5	R7
Wi-Fi?	X	S	S	N	N
Carregando?	X	S	N	S	N
Bateria > 20%?	N	S	S	S	S
Backup		X	X	X	
Não	X				X

Ra colapsa as quatro regras com bateria baixa (R2, R4, R6, R8): sempre Não, Wi-Fi e carregando não importam. As quatro regras restantes não reduzem mais: como o resultado depende do OU entre Wi-Fi e carregando, cada combinação continua sendo necessária. Redução: 8 regras viram 5, não 4 como no exemplo só com E.

Exercícios

Validação de Senha

Exercício 1

Um sistema de cadastro considera uma senha válida quando:

- Tem entre 8 e 20 caracteres
- Contém ao menos uma letra maiúscula e um dígito numérico
- Não contém espaços

Uma entrada que não seja do tipo texto deve ser rejeitada com um erro.

Identifique todas as classes de equivalência dessa especificação, válidas e inválidas, e escolha um valor representante para cada uma.

Desconto por Quantidade

⌕ Código

```
def calcular_desconto(qtd):  
    if not isinstance(qtd, int):  
        raise TypeError("Quantidade deve ser inteira")  
    if qtd <= 0:  
        raise ValueError("Quantidade deve ser positiva")  
    if qtd < 5:  
        return 0.00  
    if qtd <= 20:  
        return 0.10  
    return 0.20
```

Desconto por Quantidade

Exercício 2

Especificação:

- 1 a 5 itens: sem desconto
- 6 a 20 itens: 10% de desconto
- 21 ou mais itens: 20% de desconto
- Quantidade zero, negativa ou não inteira: erro

A implementação do slide anterior tenta seguir essa regra, mas retorna um desconto diferente do especificado para uma quantidade específica.

Usando a análise de valor-limite das fronteiras entre as faixas, identifique qual quantidade recebe o desconto errado e explique por quê.

- Teste exaustivo é inviável: o número de combinações cresce rápido demais
- Classes de equivalência particionam a entrada em grupos que se comportam de forma similar, um representante por classe
- Defeitos se concentram nas fronteiras: teste sempre os seis valores $A-1$, A , $A+1$, $B-1$, B , $B+1$
- As duas técnicas se combinam: representante do meio mais valores-limite de cada classe
- Quando o comportamento depende da combinação de várias condições, a tabela de decisão enumera as regras, e don't care reduz o que sobra de redundante

Simulação e Teste de Software (CC8550)

Aula 6 – Técnicas Caixa-Preta: Partição de Equivalência e Valor-Limite

Prof. Luciano Rossi

Ciência da Computação
Centro Universitário FEI

2º Semestre de 2026